COMPGame

COMP3423 · FLIP-LEARNING STUDY

Researcher / PI Runbook

A walkthrough of the four things only the study owner does — export the data, grade the short answers blind, generate the tutorial briefs, and read the study arms. Written for the person running the experiment, not the person who built it.

AUDIENCE
Principal investigator

YOU'LL NEED
A terminal on the server

RUN EVERYTHING FROM
FYP_Submission/

GROUNDED IN
backend/ source, this build

Start here

The **teacher panel** at `/admin` runs the class, and it deliberately cannot see answers, scores, or who is in which study condition. Everything in **this** document is the other half — the study-owner work — and it lives at the command line on the server **on purpose**: keeping it off the web is what keeps the grading blind and the real student IDs on the box.

You don't need to be a developer — but you do need a terminal

Every step below is one line you copy, paste into **PowerShell on the study server**, and run. Anything in `<angle brackets>` is a placeholder you replace. Nothing here changes a student's experience; it only reads and processes what they've already done.

The study in one line

Within-subjects, 13 topics × ~300 students. Each topic is **FLIP** or **CONTROL** per participant, assigned deterministically. Your primary outcome is Hake's normalized gain ⟨g⟩ from a uniform pre/post (Form A/B); the secondary measures are IMI, CoI, ARCS, and Paas.

Your three moments in the study

You touch this runbook at three points, not continuously. Here's the whole job on one screen — the number on each task is the card further down.

PHASE 1

Before the term

Set up once, then leave alone

1. Protect the secret key (1)
2. Sanity-check the schedule & arms (2)

PHASE 2

While the term runs

As lectures happen

1. Generate tutorial briefs (3)
2. Turn on questionnaires (4)
3. Erase a participant on request (5)

PHASE 3

At analysis time

Once data is in

1. Export the data (6)
2. Grade the short answers (7)
3. Derive the measures (8)

"I want to…" — jump to the right task

IF YOU WANT TO…	GO TO
Get all the data out as a spreadsheet	Task 6
Score the written answers fairly	Task 7
Build the pre/post gain table for each student	Task 8
Give a lecturer a summary of their tutorial group	Task 3
Switch on the IMI / CoI / ARCS questionnaires	Task 4
Remove someone who has withdrawn	Task 5
Check who is FLIP vs CONTROL	Task 2
Understand a term you don't recognise	Glossary, below

The words this runbook uses

Five terms carry the whole study. If these are clear, the rest reads easily.

Arm — FLIP or CONTROL

Which condition a student got for a given topic. **FLIP** = they played the learning game before the check; **CONTROL** = they didn't. Assigned automatically, half-and-half.

Pseudonym

A scrambled stand-in for a student ID that goes into every export. It's stable, so you can still link one student's rows together — but it can't be reversed back to a real ID off the box.

Normalized gain ⟨g⟩

Your headline result. How much of the **available** improvement a student actually made from pre-test to post-test — fairer than raw score gain because it accounts for starting point.

Cohen's κ (kappa)

How well the machine grader and a human agree, beyond chance. Above ~0.6 you can trust the machine grades; below it, treat them as colour, not data.

The sink

The single database file that quietly records every study event — a check submitted, a game finished, a questionnaire answered. Everything you export comes from it.

The three brief copies

Every tutorial brief is written three times over — one safe to project, one with student IDs for the lecturer, one with the study arms for you. Never mix them up.

PHASE 1 · BEFORE THE TERM

Set up once, then leave it alone

Two things to get right before any real student data exists.

1 Protect the secret key

BEFORE THE TERM

One tiny file, `backend/.participant_secret`, is what scrambles student IDs into pseudonyms — and it's the **only** thing that lets you join a student's pre-test to their post-test across separate exports. It loads from the `PARTICIPANT_HMAC_SECRET` environment variable, then that file, and if neither exists it makes a fresh one (with a "back this up" warning).

Lose or change it and the study quietly comes apart

A different secret makes different pseudonyms for the same students, so old and new exports can never be linked again — and nothing errors to tell you. **Copy this file off the machine before the first real run**, and after copying it onto the deployment box, verify its fingerprint (see the installation guide). It must stay **identical** for the entire study.

2 Sanity-check the schedule and arms

BEFORE THE TERM

A student's condition isn't stored anywhere — it's **computed** from their ID and the topic order, so it survives a reload, a new laptop, or a server restart and can never drift. Whether they **actually** played the game before the check is a separate thing, read from the timestamps. Before the term opens, look at both without touching any data.

QUANTITY	WHERE IT COMES FROM	WHAT IT MEANS
`arm`	assigned	FLIP or CONTROL — computed, deterministic
`played_first`	observed	did the activity happen before the post-check?
`complied`	derived	did what they did match what they were assigned?

› RUN THIS

```
# see the release windows + a sample of arm assignments
python backend\schedule.py --preview

# check the calendar for problems — run before a term starts
python backend\schedule.py --validate
```

YOU'LL SEE

`--preview` prints when each topic opens and a few example FLIP/CONTROL assignments. `--validate` is your green light: it exits quietly if the calendar is clean, and stops with **exit 1** and a message if a lecture lands somewhere it shouldn't.

PHASE 2 · WHILE THE TERM RUNS

The work that happens as lectures go by

Nothing here is urgent day-to-day — reach for it when a lecturer asks, when the ethics amendment lands, or when a student withdraws.

3 Generate tutorial briefs

DURING THE TERM

A tutorial brief is a short summary a lecturer can use in a tutorial: how the group did on the pre/post checks, how many played the game, and a plain-language read of the common sticking points. Every number is counted in code; only the free-text summary is written by the model (skip even that with `--no-llm`). It always writes **three copies**, and telling them apart matters:

COPY	WHAT'S IN IT	WHO IT'S FOR
`-discussion`	anonymised, no IDs	**Safe to project.** The script even checks it for leaked IDs.
`-teacher`	student IDs, no arms	The lecturer's eyes only. Stays on the machine.
`-research`	IDs **and** FLIP/CONTROL	You only. Never reachable through the web panel, by design.

› RUN THIS

```
python backend\generate_tutorial_report.py --topic webers-law --section B
# add --no-llm to skip the model and get numbers only
```

YOU'LL SEE

three Markdown files land under `reports/{cohort}/section-B/`. The `-discussion` and `-teacher` copies are what the course-team panel surfaces; the `-research` copy is written to disk for your analysis and is the only one that names study arms.

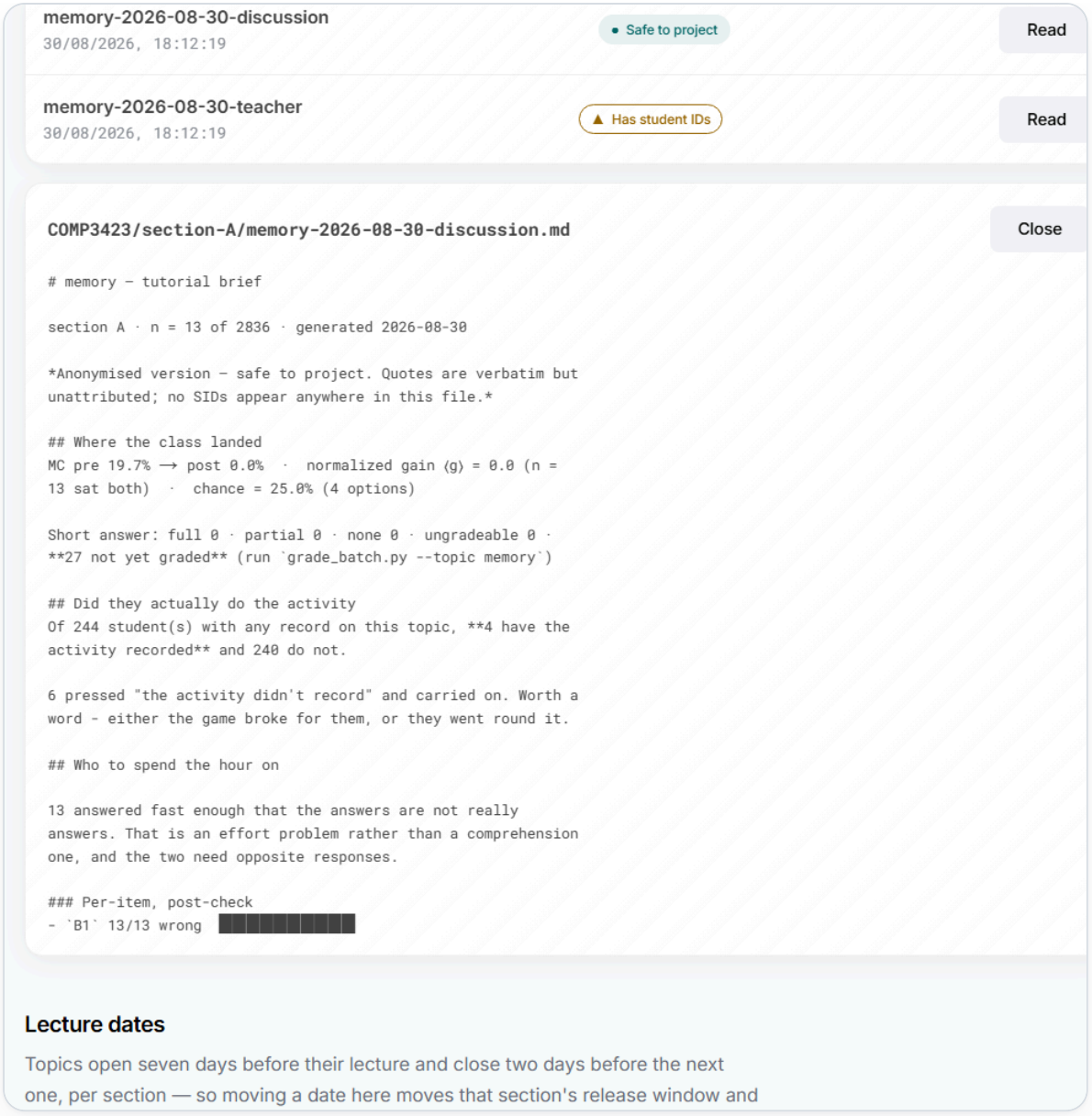

FIGURE 1 A generated brief opened in the panel — the anonymised `-discussion` copy. The scores here are counted in code; only the plain-language read is written by the model.

4 Turn on the questionnaires

DURING THE TERM

The four instruments — IMI, CoI, ARCS, and Paas — are built and wired, but **switched off** until you enable them. While off, both routes return **404**, so the app never shows a form that can't be submitted. Enable them only **after** the HSESC ethics amendment is approved.

› RUN THIS — ONLY AFTER ETHICS APPROVAL

```
# default is off (returns 404); this switches the battery on
$env:QUESTIONNAIRES_ENABLED = "1"
```

- **Consent still gates it** — before a student consents, they get the same refusal as every other recorded path.
- **One submission each** — the database itself blocks a second attempt, so no one can answer twice.
- **The scoring key never leaves the box** — students receive only the questions; you score from the study-pack codebook at analysis time.

> **Don't switch this on ahead of approval**
> It's tied to the ethics amendment, the same discipline as the telemetry flag. Enabling early collects data you're not yet cleared to collect.

5 Erase a participant on request

DURING THE TERM

When a student withdraws in the app, their account is already tombstoned and their sessions ended. If they also ask for their research data removed, this purges their rows from the sink. Do a dry run first — it counts what would go without changing anything.

› RUN THIS

```
# dry run — reports the count, changes nothing
python backend\research_store.py --forget 24E00123A

# actually erase — this cannot be undone
python backend\research_store.py --forget 24E00123A --yes
```

YOU'LL SEE

the number of rows found (dry run) or removed. This clears the research sink only; the account tombstone is handled separately and left in place.

PHASE 3 · AT ANALYSIS TIME

Getting the data out and ready to analyse

The three steps you run once the study data is in, in order: export, grade, derive.

6 Export the data

ANALYSIS TIME

The export is **always pseudonymised** — there is no "with real IDs" mode — and it automatically leaves out anyone who withdrew. It's protected by a token and **fails safe**: with no token set, it refuses to hand anything over rather than risk a leak.

› RUN THIS

```
# 1. set a token for this session (no token => the endpoint refuses, by
design)
$env:EXPORT_TOKEN = "<a long random string>"

# 2. download the full event log as a CSV spreadsheet
curl -H "X-Export-Token: $env:EXPORT_TOKEN"
"https://<host>/api/research/export?format=csv" -o events.csv

# optional: a quick "is it alive?" check — open, no token, no identifiers
curl "https://<host>/api/research/summary"
  → {"total_events": N, "participants": M}
```

YOU'LL GET

a CSV where each row is one study event, with these columns:
`id, participant_id, event_type, topic_id, mode, score, played_understanding_first, duration_ms, client_ts, server_ts, meta`

- `participant_id` is the pseudonym — never a real student ID.
- `meta` holds extras as JSON: the corpus/app version stamped on each event, and the raw questionnaire answers.

7 Grade the short answers — blind

ANALYSIS TIME

Written answers are graded with the identity and every label physically stripped out first: the ID, form, pre/post phase, score, and arm are removed, each answer is re-tagged with an anonymous code, and the whole set is shuffled so the pre/post order is gone too. The grader genuinely can't tell whose answer it is or whether it's a pre-test or a post-test.

› RUN THIS

```
# preview what would be graded — no model is called
python backend\grade_batch.py --dry-run

# grade one topic (about 15s per answer on the dev GPU)
python backend\grade_batch.py --topic webers-law

# pick up where you left off after an interruption
python backend\grade_batch.py --resume
```

YOU'LL GET

grades written to `reports/grades/`. The full set of options is `--topic`, `--seed`, `--dry-run`, `--resume`, `--sample-for-human`, `--kappa`, and `--out`.

Never lower the model's answer-length cap

It's pinned at `GRADE_NUM_PREDICT=1536` for a reason: measured on this model, a lower cap makes it return an **empty string** — which looks exactly like "couldn't grade this" — so too low a value silently turns every answer into a missing datum while the run **looks** fine. Leave it at 1536.

Check the grader against a human (Cohen's κ)

Before you trust the machine grades, hand-code a sample and compare. The coding sheet has **no machine grades in it**, so you're not anchored by what the machine thought.

› RUN THIS

```
# 1. a blank coding sheet of 60 answers for a human to grade
python backend\grade_batch.py --sample-for-human 60

# 2. once it's filled in, compare and print agreement + kappa
python backend\grade_batch.py --kappa human-coding-sheet-60.csv
```

YOU'LL SEE

the count, raw agreement, and κ. If κ comes back **below 0.6**, the tool warns you to treat the machine grades as "descriptive colour only" — report them that way, and lean on the human coding.

8 Derive the measures

ANALYSIS TIME

This turns the raw event log into the analysis-ready table — one row per student per topic — with no server and no model. It reads the database directly.

› RUN THIS

```
# the per-(student, topic) table + a coverage summary
python backend\measures.py

# zoom in on one participant
python backend\measures.py --participant 22074221D
```

YOU'LL GET

each row carrying the `arm`, whether they `played_first` (with a reason when it's unknown), whether they `complied`, the three scores (pre / post / assessment), the step timestamps, and any skip flags. The coverage summary is your headline: how many pairs you have, how many are usable, how many complied, and where the gaps are. The gain ⟨g⟩ itself is computed in the brief generator (Task 3), over students who sat both checks.

IF SOMETHING LOOKS WRONG

The five silent failures — and what each one really means

These don't throw obvious errors, so they're worth knowing on sight. In most cases the "failure" is a safety default doing its job.

WHAT YOU SEE	WHAT IT MEANS & THE FIX
Export refuses (503)	The export token isn't set. That's the fail-safe default, not a bug — set `EXPORT_TOKEN` and re-run.
Every grade came back empty	The answer-length cap was lowered below 1536, so the model returns nothing and it reads as "no data". Restore `GRADE_NUM_PREDICT=1536`.
Pseudonyms don't line up across exports	The secret key changed between runs. There's no recovery — this is why Task 1 says protect that file.
A file with IDs left the machine	Only the `-discussion` brief is safe to share. The `-teacher` and `-research` copies carry identifiers (and the research one, arms).
Questionnaires return 404	They're switched off. Correct until the ethics amendment lands — then set `QUESTIONNAIRES_ENABLED=1`.

> **Where the rest of it lives**
> Starting and stopping the server, backups, the go-live gate, and deployment are in `docs/runbook.md` and the installation guide. This document covers only the research operations layered on top.

COMPGame · COMP3423 Human–Computer Interaction · Researcher / PI Runbook · Grounded in the backend source of this build.